

Tool implementation — snapshot.py

src/lab_billing/forecast/snapshot.py — git blob 3f9d31fba1c0 — v1.4.0 (2026-09-22)

```
"""Snapshot construction and event labeling (module B, spec sections 11-13).

Landmark t = first observable submission + 7 days (study default). Features
are computed strictly from data with available_at <= t. Outcomes are the
first observable positive payment allocation, first no-payment closure, or
still-open at horizon H=30 days. Observations that end before H without an
event are right-censored and never labeled as negatives.

Leakage guards (T13): hidden truth ids, future states, final payment
amounts and test labels are structurally excluded from feature vectors.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from typing import Any

_TYPE_PRIO = {"PAYMENT_POSTED": 0, "CLOSED_NO_PAYMENT": 1}

PAYER_CATEGORY = {
    "AET": "commercial", "UHC": "commercial", "HUM": "commercial",
    "BCB": "commercial", "CIG": "commercial",
    "MCR": "government", "MCD": "government",
    "SELF": "self", "": "unknown",
}

@dataclass
class Snapshot:
    snapshot_id: str
    lineage_id: str
    org_token: str
    t: datetime
    max_input_available_at: datetime
    features: dict[str, Any]
    outcome: str # payment / close_no_payment / still_open / censored / ambiguous
    outcome_interval: int | None # interval index of first event (0-based) or None
    label_known: bool

def _utc(dt) -> datetime:
    if isinstance(dt, str):
        s = dt.strip()
        if s.endswith("Z"):
            s = s[:-1] + "+00:00"
        dt = datetime.fromisoformat(s)
    return dt.astimezone(timezone.utc)

class SnapshotBuilder:
    def __init__(self, landmark_days: int = 7, horizon_days: int = 30,
```

```

        interval_days: int = 5, as_of: str | None = None):
    self.landmark_days = landmark_days
    self.horizon_days = horizon_days
    self.interval_days = interval_days
    self.as_of = _parse(as_of) if as_of else None

def build(self, lineages: list[dict], events: list[dict],
        allocations: list[dict]) -> list[Snapshot]:
    snaps: list[Snapshot] = []
    ev_by_lineage: dict[str, list[dict]] = {}
    # payments carry empty bill_record_id and attach via allocations;
    # one payment may span several lineages (C-04: no overwrite)
    alloc_lineage: dict[str, set[str]] = {}
    # earliest allocation knowable time per payment (C-05)
    alloc_at: dict[str, str] = {}
    for a in allocations:
        lin = _lineage_of_record(a.get("bill_record_id", ""), lineages)
        if lin:
            alloc_lineage.setdefault(a["payment_event_id"], set()).add(lin)
            aa = a.get("available_at")
            if aa and (a["payment_event_id"] not in alloc_at or aa <
alloc_at[a["payment_event_id"]]):
                alloc_at[a["payment_event_id"]] = aa

    def _eff_at(e: dict):
        when = _utc(e["available_at"])
        if e["event_type"] == "PAYMENT_POSTED" and e["event_id"] in alloc_at:
            when = max(when, _utc(alloc_at[e["event_id"]]))
        return when

    def _sort_key(e: dict):
        return (_eff_at(e), _TYPE_PRIO.get(e["event_type"], 2), e["event_id"])

    for e in events:
        lin = _lineage_of(e, lineages)
        targets = [lin] if lin else sorted(alloc_lineage.get(e["event_id"], set()))
        for tl in targets:
            ev_by_lineage.setdefault(tl, []).append(e)

    n_intervals = max(1, self.horizon_days // self.interval_days)
    for lin in lineages:
        lid = lin["lineage_id"]
        evs = sorted(ev_by_lineage.get(lid, []), key=_sort_key)
        if not evs:
            continue
        first_sub = next((e for e in evs if e["event_type"] == "SUBMITTED"), None)
        if first_sub is None:
            continue
        t = _utc(first_sub["available_at"]) + timedelta(days=self.landmark_days)
        if self.as_of and t > self.as_of:
            continue # landmark beyond analysis horizon
        known = [e for e in evs if _eff_at(e) <= t]
        future = [e for e in evs if _eff_at(e) > t]
        for e in future:
            e["_eff_at"] = _eff_at(e)
        # C-07: outcomes already determined before the landmark are not
        # prediction targets
        if any((e["event_type"] == "PAYMENT_POSTED"

```

```

        and (int(e.get("amount_minor") or 0) > 0))
        or e["event_type"] == "CLOSED_NO_PAYMENT"
        for e in known):
            continue
        feats = self._features(lin, known, first_sub, t)
        outcome, interval = self._label(lin, future, t, n_intervals)
        snaps.append(Snapshot(
            snapshot_id=f"{lid}@t_{ts(t)}",
            lineage_id=lid, org_token=lin.get("org_token", ""),
            t=t, max_input_available_at=max((_eff_at(e) for e in known),
                                           default=t),
            features=feats, outcome=outcome,
            outcome_interval=interval,
            label_known=outcome in ("payment", "close_no_payment", "still_open"),
        ))
    return snaps

# ----- features
def _features(self, lin: dict, known: list[dict], first_sub: dict,
              t: datetime) -> dict:
    t_dt = _utc(t)
    sub_at = _utc(first_sub["available_at"])
    age_days = max(0.0, (t_dt - sub_at).total_seconds() / 86400)
    corrected = sum(1 for e in known if e["event_type"] == "CORRECTED")
    voids = sum(1 for e in known if e["event_type"] == "VOID_VERSION")
    payments_pre = sum(1 for e in known if e["event_type"] == "PAYMENT_POSTED")
    unknown_pre = sum(1 for e in known if e["event_type"] == "UNKNOWN")
    status_pre = sum(1 for e in known if e["event_type"] == "STATUS_UPDATED")
    closes_pre = sum(1 for e in known if e["event_type"] == "CLOSED_NO_PAYMENT")
    reopens_pre = sum(1 for e in known if e["event_type"] == "REOPENED")
    n_members = len(lin.get("record_ids", "").split("|")) if lin.get("record_ids")
else 1
    quality_penalty = (
        int(lin.get("conflict_count", 0) or 0)
        + int(lin.get("review_count", 0) or 0)
    )
    return {
        "age_days": round(age_days, 2),
        "payer_category": lin.get("payer_category", "unknown"),
        "billed_amount_minor": int(lin.get("billed_amount_minor", 0) or 0),
        "n_versions": n_members,
        "n_corrected": corrected,
        "n_voids": voids,
        "n_payments_pre_t": payments_pre,
        "n_unknown_pre_t": unknown_pre,
        "n_status_updates_pre_t": status_pre,
        "n_closes_pre_t": closes_pre,
        "n_reopens_pre_t": reopens_pre,
        "quality_penalty": quality_penalty,
        "coverage_status": lin.get("coverage_status", "complete"),
    }

# ----- label
def _label(self, lin: dict, future: list[dict], t: datetime,
           n_intervals: int) -> tuple[str, int | None]:
    """First observable outcome within H days after t."""
    t_dt = _utc(t)
    H_end = t_dt + timedelta(days=self.horizon_days)

```

```

# future list arrives already sorted deterministically
for e in future:
    when = e["_eff_at"] if "_eff_at" in e else _utc(e["available_at"])
    if when >= H_end:
        break # horizon endpoint excluded (C-06: no 7th interval)
    if e["event_type"] == "PAYMENT_POSTED":
        if e.get("amount_minor") is not None and int(e.get("amount_minor") or 0) >
0:
            iv = math.floor((when - t_dt).total_seconds() / (self.interval_days *
86400))
            return "payment", max(0, min(iv, n_intervals - 1))
        elif e["event_type"] == "CLOSED_NO_PAYMENT":
            iv = math.floor((when - t_dt).total_seconds() / (self.interval_days *
86400))
            return "close_no_payment", max(0, min(iv, n_intervals - 1))
    # no event within H: still open only if reliably observed to H
    if self._observed_through(lin, t_dt, H_end):
        return "still_open", None
    return "censored", None

def _observed_through(self, lin: dict, start: datetime, end: datetime) -> bool:
    cov = lin.get("coverage_status", "complete")
    if cov != "complete":
        return False
    ce = lin.get("coverage_end")
    if ce is None:
        return False
    try:
        return _utc(ce) >= end
    except (TypeError, ValueError):
        return False

def _lineage_of(event: dict, lineages: list[dict]) -> str | None:
    rid = event.get("bill_record_id", "")
    return _lineage_of_record(rid, lineages)

def _lineage_of_record(rid: str, lineages: list[dict]) -> str | None:
    if not rid:
        return None
    for lin in lineages:
        if rid in lin.get("record_ids", "").split("|"):
            return lin["lineage_id"]
    return None

def _parse(s: str) -> datetime:
    if s.endswith("Z"):
        s = s[:-1] + "+00:00"
    return datetime.fromisoformat(s).astimezone(timezone.utc)

def _ts(dt: datetime) -> str:
    return dt.strftime("%Y%m%d")

```